

Real-Time Air Quality Analytics Pipeline

Bogdan Dumitrescu & Marinescu Alexandru

Objective

We aim to design and implement a scalable, real-time stream processing architecture that monitors global air quality. The system will ingest live telemetry from public environmental sensors, process the data on the fly to eliminate noise, and perform windowed aggregations. Ultimately, the project aims to detect severe pollution anomalies (e.g PM2.5 and CO2 spikes) in real-time, generate automated alerts, and provide short-term predictive forecasting through an interactive web application.

Data Sources / Datasets

- **OpenAQ API:** We will utilize the Open Air Quality API as our primary open-source data provider. It delivers live, harmonized air quality data (PM2.5, PM10, O3, CO2) aggregated from government-grade monitoring stations worldwide. Data will be fetched continuously in JSON format to simulate a high-throughput sensor network.

Processing Engines Used

- **Apache Kafka:** Acts as the distributed message broker. It will decouple the data ingestion phase from the processing phase, handling the high-throughput stream of raw environmental data.
- **Apache Flink (PyFlink):** Acts as the core stream processing engine. It will consume live data from Kafka to perform stateful computations, specifically time-based windows to calculate moving averages and identify actionable pollution trends.
- **Scikit-Learn (Python):** Used as the analytics engine to apply lightweight linear regression on the processed stream for short-term pollution forecasting.

Application Architecture & Data Flow

To achieve our objective, the system logic is divided into four continuous, real-time stages:

1. **Continuous Ingestion (The Producer):** A Python microservice continuously polls the OpenAQ API for specific geographic coordinates. It normalizes the raw payload and publishes the telemetry data to a Kafka topic.
2. **Stream Processing & Anomaly Detection (The Engine):** An Apache Flink job subscribes to the producer topic. It applies a 1-minute Tumbling Window to calculate the moving average of PM2.5 levels, effectively smoothing out transient sensor noise. If the moving average breaches predefined safety thresholds, Flink generates an anomaly event and publishes it to a secondary Kafka topic.

3. **Predictive Analytics:** Concurrently, a predictive model analyzes the recent time-series data from the processed stream to forecast the expected pollution trajectory for the upcoming hour.
4. **Presentation Layer (The Web App):** A frontend web dashboard acts as the consumer. It reads the clean data and alerts from Kafka to render a live, interactive UI. The dashboard displays real-time time-series charts of pollution trends, an active alert log for health warnings, and the machine learning-driven forecasts.